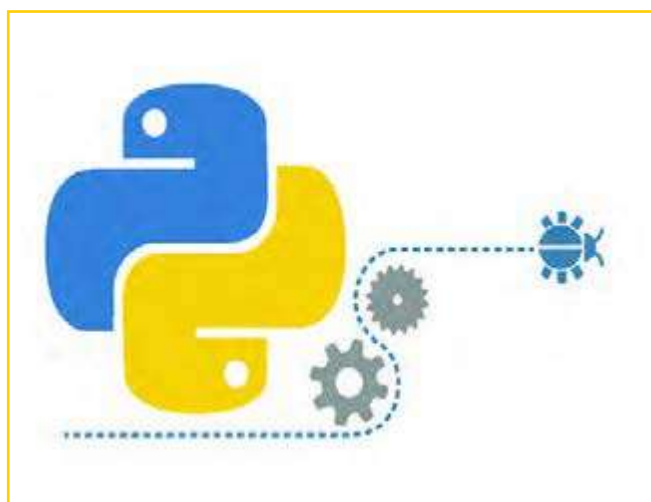


برنامه‌نویسی پایتون (۳)



در سال‌های گذشته در پودمان‌های برنامه‌نویسی پایتون ۱ و ۲ با برخی از مفاهیم ساده و مقدماتی زبان پایتون و همچنین کتابخانه‌های turtle و pygame آشنا شدید. برای آشنایی بیشتر با کتابخانه‌های پایتونی مانند کتابخانه tkinter، ابتدا باید شناخت بیشتری از مبحث شیء‌گرایی داشته باشیم. در این درس، پس از بررسی و تکمیل مبحث تابع، با مباحث شیء‌گرایی و رابط کاربر گرافیکی آشنا می‌شویم. آشنایی با این مباحث به شما کمک می‌کند تا بتوانید برنامه‌های تحت ویندوز بنویسید.

برخی از شایستگی‌هایی که در این پودمان به دست می‌آورید :

- کار گروهی، مسئولیت‌پذیری، مدیریت منابع، فناوری اطلاعات و ارتباطات؛
- کسب مهارت‌های حل مسئله و پرورش تفکر رایانشی؛
- آشنایی با محیط برنامه‌نویسی visual studio code؛
- آشنایی با مفاهیم *args و **kwargs؛
- آشنایی با مفهوم scope یا namespace (فضای نام)؛
- آشنایی با توابع داخلی پایتون (built-in method)؛
- آشنایی با مبحث شیء‌گرایی در پایتون؛
- آشنایی با رابط کاربر گرافیکی در پایتون.



قبل از شروع قسمت سوم برنامه‌نویسی پایتون، پیشنهاد می‌شود با اسکن رمزینه فیلم‌های آموزشی آنچه را که در سال‌های گذشته در زمینه برنامه‌نویسی پایتون آموختید، مشاهده کنید.

مقدمه و یادآوری

همان طور که در سال گذشته دیدیم فرم کلی تعریف تابع به صورت زیر است :

def نام تابع (پارامترهای ورودی تابع) :

دستورات بدنه تابع

return مقدار خروجی تابع

و از دستور return برای بازگرداندن یک مقدار از تابع استفاده می شود. بعد از آنکه اجرا شد، از تابع خارج می شویم و دستورات بعد از آن اجرا نمی شود.

همچنین شکل کلی فراخوانی تابع برای نمایش خروجی به صورت زیر است :

print (آرگومان های ورودی) نام تابع)

برای آشنایی با نوشتن تابع بیشینه سه عدد ورودی، رمزینه را اسکن کنید.



مثال ۱: تابعی بنویسید که ۳ عدد را دریافت کرده و عدد بزرگ تر را return کند.

پاسخ : تصویری از برنامه مورد نظر در شکل زیر آمده است که در ادامه هر یک از خطوط این برنامه بررسی شده است.

```
1 def maximum(x,y,z):
2     max=x
3     if y>max:
4         max=y
5     if z>max:
6         z=max
7     return max
8
9 print('max=',maximum(10,20,15))
```

```
>>>
= RESTART: C:/Users/3051273120/AppData
/Local/Programs/Python/Python311/max.p
y
max= 20
```

در خطوط ۱ تا ۷ این برنامه، تابعی به نام maximum با سه پارامتر ورودی x و y و z تعریف شده است. فرض شده که x بزرگ ترین عدد است؛ به همین دلیل آن را توسط عملگر انتساب در متغیر max ذخیره کرده ایم. حال متغیرهای y و z را با max مقایسه می کنیم. هر کدام که بزرگ تر بود، در متغیر max ذخیره می شود. سرانجام متغیر max را توسط دستور return برمی گردانیم. در خط ۹ توسط دستور print تابع را فراخوانی کرده و آرگومان های ۱۰ و ۲۰ و ۱۵ را به آن ارسال می کنیم (به اصطلاح گفته می شود که آرگومان ها را به تابع پاس می دهیم) و مقدار ۲۰ در خروجی چاپ می شود.

کار کلاسی

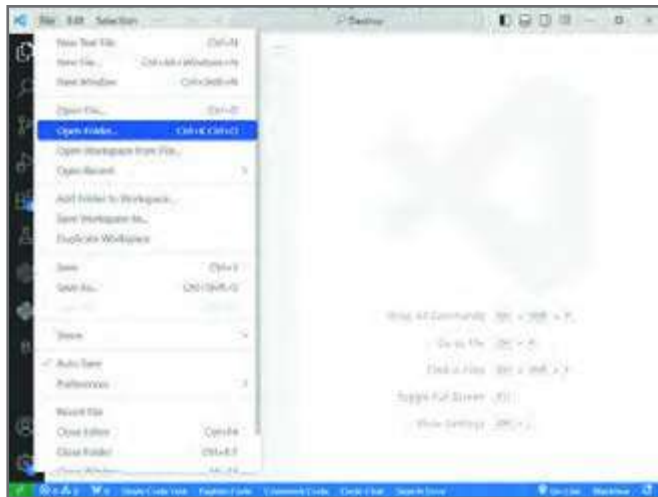


در مثال بالا خط شماره ۹ را توسط تابع F-string بنویسید.

در سال‌های گذشته از محیط IDLE، که محیطی ساده و ابتدایی برای کدنویسی است استفاده کردیم. در این درس از محیط نرم‌افزار Visual Studio Code برای برنامه‌نویسی استفاده خواهیم کرد. برنامه‌نویسی در محیط vscode خیلی راحت‌تر و سریع‌تر انجام می‌شود. لازم است توجه کنید که برای استفاده از نرم‌افزار Visual Studio Code باید برنامه IDLE همچنان روی رایانه شما نصب باشد. پس از ورود به برنامه vscode در ابتدا یک folder به نام myPrograms می‌سازیم (شکل ۴-۱-الف) و وارد آن می‌شویم و با کلیک بر روی گزینه new file یک فایل پایتونی با پسوند .py می‌سازیم (شکل ۴-۱-ب). برای آن که بتوان برنامه‌های پایتون را در محیط Visual Studio Code اجرا کرد افزونه‌های python و python extension pack و pylance را از سربرگ افزونه‌ها نصب می‌کنیم (شکل ۴-۲).



(ب)



(الف)

شکل ۴-۱



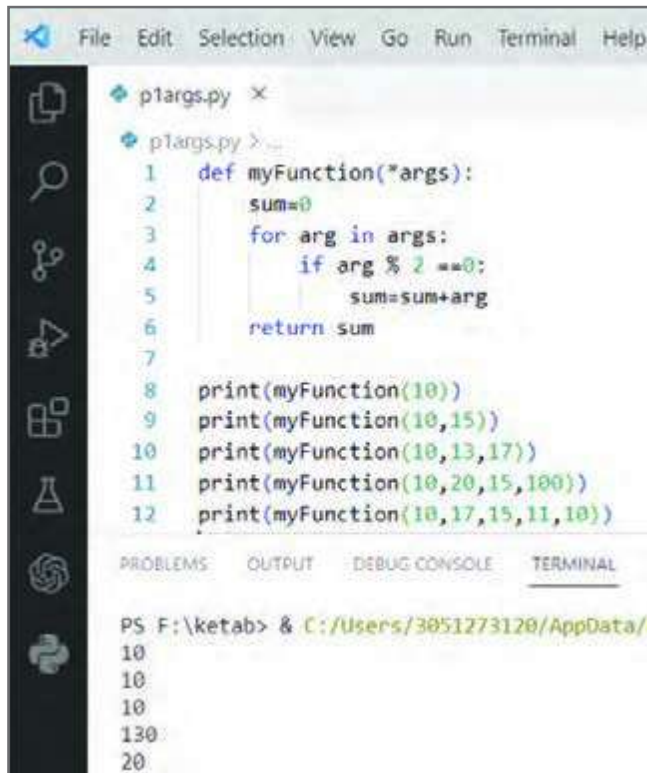
برای آشنایی با محیط
Visual Studio Code
رمزین را اسکن کنید.

شکل ۴-۲

آشنایی با مفهوم *args

هرگاه بخواهیم تابعی بنویسیم که مقدارهای مختلف از ورودی (value) را بتواند دریافت کند، از *args به عنوان پارامتر ورودی استفاده می‌کنیم. برای آشنایی بیشتر با مفهوم *args به مثالی که در ادامه آمده است توجه کنید.

مثال ۲: تابعی بنویسید که از بین اعداد ارسال شده به آن، مجموع اعداد زوج را محاسبه کند و برگرداند. سپس تابع را فراخوانی کنید.



```
File Edit Selection View Go Run Terminal Help
ptargs.py x
ptargs.py > ...
1 def myFunction(*args):
2     sum=0
3     for arg in args:
4         if arg % 2 ==0:
5             sum=sum+arg
6     return sum
7
8 print(myFunction(10))
9 print(myFunction(10,15))
10 print(myFunction(10,13,17))
11 print(myFunction(10,20,15,100))
12 print(myFunction(10,17,15,11,10))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS F:\ketab> & C:/Users/3051273120/AppData/Local/Programs/Python/Python38-32/Python.exe ptargs.py
10
10
10
130
20
```

پاسخ: تصویری از برنامه مورد نظر در شکل روبه‌رو آمده است که در ادامه هر یک از خطوط این برنامه بررسی شده است.



برای آشنایی با *args، رمزینه را اسکن کنید.

همان طور که در شکل بالا دیده می‌شود در خطوط ۱ تا ۶ تابع myFunction تعریف شده است که داخل پرانتز و قبل از ورودی *args علامت ستاره قرار دارد. با این کار تابع می‌تواند ۰، ۱، ۲، ۳ یا هر تعداد آرگومان ورودی را دریافت کند. در خطوط شماره ۸ تا ۱۲ به ترتیب تابع myFunction را با ۱، ۲، ۳، ۴ و ۵ ورودی فراخوانی کردیم. در برنامه بالا به جای کلمه args می‌توان از اسامی دیگری هم برای پارامتر ورودی تابع استفاده کرد، ولی برنامه‌نویسان پایتون همیشه از کلمه args استفاده می‌کنند.

آشنایی با مفهوم **kwargs

اگر بخواهیم تابعی بنویسیم که بتواند هر تعداد کلید : مقدار (key: value) را دریافت کند از **kwargs به عنوان پارامتر ورودی تابع استفاده می‌کنیم. برای آشنایی بیشتر با این موضوع به مثالی که در ادامه آمده است توجه کنید.

مثال ۳: تابعی بنویسید که هر تعداد کلید و مقدار به آن ارسال شد، آن‌ها را چاپ کند.

پاسخ: تصویری از برنامه مورد نظر در شکل صفحه بعد آمده است که در ادامه هر یک از خطوط این برنامه بررسی شده است. در برنامه صفحه بعد در خط ۲ از تابع items() استفاده شده است. همان طور که در سال گذشته دیدید کلمه item به معنی key: value (کلید : مقدار) است.



برای آشنایی با مفهوم `**kwargs` رمزینه را اسکن کنید.

همان طور که در شکل بالا دیده می شود، هنگام فراخوانی تابع `myFunction` در خطوط ۵ تا ۷ به ترتیب ۱، ۲ و ۳ کلید - مقدار به آن ها ارسال شده است. هر چند داخل تابع (خط ۲) می توان به جای `key` و `value` برای متغیرها از اسامی دیگری نیز استفاده کرد، ولی برنامه نویسان پایتون همیشه از `key` و `value` استفاده می کنند.

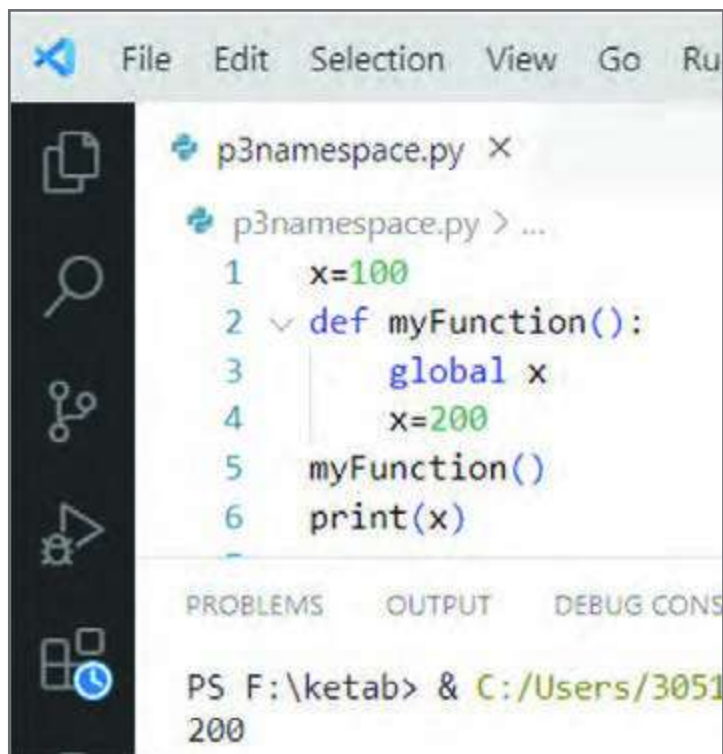
آشنایی با مفهوم `scope` یا `namespace` (فضای نام)

هرگاه در پایتون متغیری را قبل از تابع تعریف کنیم، برای آن که بتوان در داخل تابع از آن استفاده کرد، از کلمه کلیدی `global` قبل از نام متغیر استفاده می شود. لازم است توجه کنید که در برخی از زبان های برنامه نویسی، مانند سی شارپ نیازی به این کار نیست و می توان آن متغیر را به راحتی داخل تابع به کار برد. متغیری که داخل تابع تعریف می شود در فضای نام `local` (محلی) و متغیری که قبل از تابع تعریف می شود در فضای نام `global` (سراسری) قرار دارد.



برای مثال در برنامه شکل ۳-۴ یک متغیر به نام `x` با مقدار اولیه ۱۰۰ قبل از تابع تعریف شده است. همچنین یک متغیر دیگر به نام `x` و با مقدار اولیه ۲۰۰ در داخل تابع `myFunction` تعریف شده است. این دو متغیر یکسان نیستند و دو مکان مختلف در حافظه هستند که هیچ ارتباطی با هم ندارند. در خط ۵ تابع فراخوانی شده است. با نوشتن دستور `print` در خط شماره ۶، مقدار ۱۰۰ را می توان در خروجی مشاهده کرد.

حال اگر بخواهیم با فراخوانی تابع myFunction، مقدار متغیر x که قبل از تابع تعریف شده است به ۲۰۰ تغییر کند باید داخل تابع از کلمه global استفاده کنیم؛ یعنی به پایتون می‌گوییم که متغیری که در خط ۴ داخل تابع تعریف شده، همان متغیری است که در خط شماره ۱ تعریف شده بود. بنابراین با اجرای برنامه در خروجی ۲۰۰ چاپ می‌شود (شکل ۴-۴).



```

File Edit Selection View Go Run

p3namespace.py x
p3namespace.py > ...
1 x=100
2 def myFunction():
3     global x
4     x=200
5     myFunction()
6     print(x)

PROBLEMS OUTPUT DEBUG CONSOLE
PS F:\ketab> & C:/Users/3051
200

```

شکل ۴-۴



برای آشنایی با کلمه کلیدی global، رمزینه را اسکن کنید.

آشنایی با توابع داخلی پایتون (built-in method)

به طور کلی در همهٔ زبان‌های برنامه‌نویسی توابعی از قبل نوشته شده که در اختیار برنامه‌نویسان است. در جدول صفحه بعد لیست برخی از توابع داخلی پایتون آمده است.



برای آشنایی با توابع داخلی پایتون، رمزینه را اسکن کنید.

تابع	توضیحات	مثال کاربردی
abs	محاسبه قدر مطلق	<code>print (abs (-5)) ⇒ 5</code>
eval	محاسبه مقدار عددی یک عبارت ریاضی	<code>print (eval ("10+5-3*6")) ⇒ -3</code>
all	تمام اعداد مقدار منطقی True دارند. صفر مقدار منطقی False دارد. تابع all یک لیست را به عنوان ورودی دریافت کرده و در صورتی که یکی از عناصر لیست، صفر باشد خروجی False می شود.	<code>print (all ([10,15,26,35,0])) ⇒ False</code> <code>print (all ([14,12,20,14,17])) ⇒ True</code>
any	تابع any یک لیست را به عنوان ورودی دریافت کرده و در صورتی که همه عناصر لیست، صفر باشد خروجی False می شود.	<code>print (any ([10,15,26,35,0])) ⇒ True</code> <code>print (any ([0,0,0,0,0])) ⇒ False</code>
format	توسط تابع format می توان مقادیر متغیرهای name و age را در محل های مشخص شده چاپ کرد.	<code>txt1="My name is {name}, I'm{age}"</code> <code>format (name = "John", age =36)</code> <code>print (txt1)</code>
chr	کد اسکی را به عنوان ورودی دریافت می کند و کاراکتر معادل آن را چاپ می کند.	<code>print (chr (65)) ⇒ A</code> <code>print (chr (97)) ⇒ a</code>
ord	کاراکتر را دریافت می کند و کد اسکی آن را چاپ می کند.	<code>print (ord("A")) ⇒ 97</code>
round	عدد ورودی را به تعداد ارقام مشخص شده گرد می کند.	<code>print (round (5.76543,2)) ⇒ 5.77</code>
pow	دو تا ورودی عددی دریافت می کند. اولین ورودی را به توان دومین ورودی می رساند.	<code>print (Pow(4,3)) ⇒ 64</code>
id	آدرس متغیر در حافظه را مشخص می کند.	<code>x= 100</code> <code>print (id(x)) ⇒ 74436240</code>
split	یک رشته را دریافت کرده و به لیستی از کلمات تجزیه می کند. معیار تجزیه فاصله هست.	<code>s="ali is a good student"</code> <code>s.split()</code> <code>print(s) ⇒</code> <code>["ali","is","a","good","student"]</code>

آشنایی با شیءگرایی

در ابتدا در زبان هایی مانند C استاندارد، روش برنامه نویسی شیءگرایی وجود نداشت و برنامه نویسان به صورت تابع گرا برنامه نویسی می کردند، ولی امروزه از این روش، در تمامی زبان های برنامه نویسی مانند سی شارپ (C#)، سی پلاس پلاس (C++)، پایتون و... استفاده می شود؛ بنابراین ضرورت دارد که به عنوان یک برنامه نویس با آن آشنا شویم.

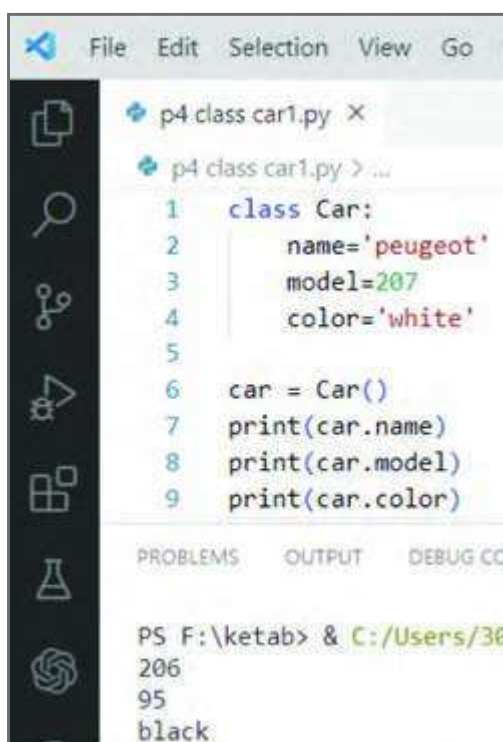


برای آشنایی بیشتر با شیءگرایی، رمزینه را اسکن کنید.

چرا شیء گرایی؟

برای پاسخ به این پرسش ابتدا باید با مفهوم کلاس آشنا شویم. کلاس شامل تعدادی متغیر و تابع است. به متغیرهایی که داخل کلاس استفاده می‌شود، در اصطلاح attribute یا ویژگی و به توابعی که داخل آن استفاده می‌شود، متد (method) گفته می‌شود. برای آنکه از کلاس استفاده کنیم، باید نمونه‌ای^۱ (شیئی)^۲ از کلاس ایجاد کنیم. در واقع کلاس، الگو یا طرحی است که ویژگی‌ها یا رفتار یک شیء را تعریف می‌کند. به عنوان مثال در کلاس میوه‌ها، نمونه‌هایی مانند پرتقال، سیب، گیلان و ویژگی‌های کلی کلاس میوه‌ها را دارند؛ در عین حال ویژگی‌های منحصر به فرد خود را هم دارند.

مثال ۴: کلاسی به نام Car را به گونه‌ای تعریف کنید که ویژگی‌های یک خودرو شامل نام، مدل و رنگ در آن وجود داشته باشد. پاسخ: همان طور که پیش از این در مبحث توابع گفته شد، هنگامی که تابعی را می‌نویسیم برای اجرای آن باید فراخوانی شود. در مورد کلاس نیز تقریباً همین گونه است ولی به جای فراخوانی باید یک نمونه (شیء) از کلاس Car ایجاد کرد (شکل زیر). برای چاپ مقادیر name و model و color قبل از نام متغیرهای داخل کلاس باید نام شیء را بنویسیم. در شکل زیر ابتدا یک نمونه یا شیء از کلاس Car ایجاد کرده‌ایم (خط ۶) و سپس برای چاپ مقادیر نام و مدل و رنگ خودرو قبل از نام متغیرها، نام شیء (car) را به همراه نقطه نوشته‌ایم (خطوط ۷ تا ۹).



```
File Edit Selection View Go
p4 class car1.py X
p4 class car1.py > ...
1 class Car:
2     name='peugeot'
3     model=207
4     color='white'
5
6 car = Car()
7 print(car.name)
8 print(car.model)
9 print(car.color)

PROBLEMS OUTPUT DEBUG CO
PS F:\ketab> & C:/Users/30
206
95
black
```

در مثال ۴ هر تعداد شیء که ایجاد کنیم، در تمام آن‌ها مقدار ویژگی name و model و color به ترتیب peugeot و 207 و white

هست.

در برنامه شکل ۵-۴ الف در خطوط ۶ و ۷ دو نمونه (شیء) از کلاس Car به نام های car1 , car2 ایجاد کرده ایم. هنگامی که مقادیر name و model و color را در شیء car2 چاپ می کنیم دوباره همان مقادیر قبلی که در مورد شیء car1 چاپ می شود، در خروجی می بینیم.

```

1 class Car:
2     def __init__(self, name, model, color):
3         self.n=name
4         self.m=model
5         self.c=color
6
7 car1=Car('pride',98,'white')
8 car2=Car('206',95,'black')
9 print(car2.n)
10 print(car2.m)
11 print(car2.c)
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS F:\ketab> & C:/Users/3051273120/AppData/Local

206
95
black

(ب)

```

1 class Car:
2     name='peugeot'
3     model=207
4     color='white'
5
6 car1 = Car()
7 car2 = Car()
8 print(car1.name)
9 print(car1.model)
10 print(car1.color)
11 print(car2.name)
12 print(car2.model)
13 print(car2.color)

```

PROBLEMS OUTPUT DEBUG CONSOLE

PS F:\ketab> & C:/Users/3051273120/AppData/Local

peugeot
207
white
peugeot
207
white

(الف)

شکل ۵-۴



برای آشنایی با تابع __init__ رمزینه را اسکن کنید.

آشنایی با تابع __init__

برای آنکه هر شیء ویژگی های (attribute) خاص خودش را داشته باشد و با بقیه اشیاء متفاوت باشد، باید از تابعی به نام __init__ در کلاس استفاده کنیم تا بتوان در هنگام ساخت یک شیء از کلاس Car مقادیر نام و مدل و رنگ را برای آن مشخص کرد. به این منظور به برنامه ای که در شکل ۵-۴ آمده است توجه کنید.

در برنامه شکل ۶-۴ داخل کلاس، تابع __init__ دارای سه پارامتر ورودی color و model و name است. بنابراین در خطوط ۷ و ۸ هنگامی که اشیاء car1 و car2 را ایجاد می کنیم، مقادیر نام، مدل و رنگ خودرو هم زمان مشخص می شوند. در اینجا مقادیر name و model و color به ترتیب وارد ویژگی های n و m و c می شود. به عبارت دیگر در زمان ساخت شیء ویژگی های آن مقادیردهی می شود (به خطوط ۷ و ۸ دقت کنید). البته بهتر است که نام ویژگی های داخل کلاس با نام پارامترهای تابع __init__ یکسان باشد. بنابراین برنامه قبلی را به صورت شکل ۶-۴ ب می نویسیم.

```

1 class Car:
2     def __init__(self, name, model, color):
3         self.name = name
4         self.model = model
5         self.color = color
6
7 car1 = Car('pride', 90, 'white')
8 car2 = Car('206', 95, 'black')
9 print(car1.name)
10 print(car1.model)
11 print(car1.color)
12 print(car2.name)
13 print(car2.model)
14 print(car2.color)
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS F:\ketab> & C:/Users/3051273120/AppData/Local/
pride
90
white
206
95
black

```

(ب)

```

1 class Car:
2     def __init__(self, name, model, color):
3         self.n = name
4         self.m = model
5         self.c = color
6
7 car1 = Car('pride', 90, 'white')
8 car2 = Car('206', 95, 'black')
9 print(car2.n)
10 print(car2.m)
11 print(car2.c)
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS F:\ketab> & C:/Users/3051273120/AppData/Local
206
95
black

```

(الف)

شکل ۴-۶

نکته:

- کلمه `init` مخفف کلمه `initialization` و به معنای مقدار دهی اولیه است.
- قبل و بعد از کلمه `init` باید از `dunder` استفاده کرد که مخفف `double underline` به معنای دوتا خط زیرین (__) است.
- تمام توابعی که داخل کلاس نوشته می شوند باید حداقل دارای پارامتر ورودی `self` باشند که اگر این پارامتر نوشته نشود پایتون خطا می دهد.
- همچنین قبل از نام ویژگی های `name` و `model` و `color` باید از نام شیء استفاده شود. از آنجا که اشیاء را خارج از کلاس ایجاد می کنیم و داخل کلاس هیچ شیئی هنوز تعریف نشده است از کلمه `self` قبل از نام ویژگی ها استفاده می شود.
- تابع `__init__` در زمان ساخت شیء خود به خود فراخوانی می شود.
- در زمان ساخت شیء باید به تعداد پارامترهای داخل تابع `__init__` آرگومان ارسال شود. در شکل ۴-۶ چون تابع `__init__` دارای ۳ پارامتر `color`, `model`, `name` است، در خطوط ۷ و ۸ که اشیاء `car1` و `car2` ایجاد شده، سه آرگومان (مقدار) هم نوشته شده است.

مثال ۵: یک کلاس به نام `Rectangle` (مستطیل) با تابع `__init__` که دارای پارامترهای `width`, `height` به ترتیب برای مقداردهی طول و عرض مستطیل اند تعریف کنید. سپس داخل کلاس، توابع `mohit` و `masahat` را برای محاسبه مقادیر آنها بنویسید. آنگاه یک شیء از کلاس `Rectangle` ایجاد کرده و متدهای مربوط به محاسبه محیط و مساحت را فراخوانی کنید.

```

1 class Rectangle:
2     def __init__(self,width,height):
3         self.width=width
4         self.height=height
5
6     def mohit(self):
7         m=(self.width + self.height)*2
8         return m
9
10    def masahat(self):
11        ma=self.width * self.height
12        return ma
13
14    R1=Rectangle(100,150)
15    print("mohit=",R1.mohit())
16    print("masahat=",R1.masahat())

```

PS F:\ketab> & C:/Users/3851273120/AppData/Local/Programs/Python/Python38-64/Python.exe

mohit= 500
masahat= 15000

پاسخ : در برنامه شکل مقابل در خط ۱ یک کلاس به نام Rectangle ایجاد کرده و داخل کلاس، تابع __init__ دو پارامتر width و height تعریف شده است. همچنین داخل کلاس، دو تابع دیگر به نام های mohit و masahat نوشته شده است.



برای آشنایی بیشتر با این مثال، رمزین را اسکن کنید.

```

1 import math
2 class Circle:
3     def __init__(self,r):
4         self.r=r
5     def masahat(self):
6         ma=self.r**2*math.pi
7         return ma
8     def mohit(self):
9         mo=self.r*2*math.pi
10        return mo
11    c1=Circle(10)
12    print('masahat=',c1.masahat())
13    print('mohit=',c1.mohit())

```

PS F:\ketab> & C:/Users/3851273120/AppData/Local/Programs/Python/Python38-64/Python.exe

masahat= 314.1592653589793
mohit= 62.83185307179586

مثال ۶: یک کلاس به نام Circle با تابع __init__ که دارای پارامتر r است تعریف کنید و برای محاسبه محیط و مساحت، توابع mohit و masahat را داخل آن بنویسید. سپس یک شیء (نمونه) از کلاس Circle ایجاد کرده و مقدار محیط و مساحت را چاپ کنید.

پاسخ : تصویری از برنامه مورد نظر در شکل مقابل آمده است که در ادامه هر یک از خطوط این برنامه بررسی شده است.



برای آشنایی بیشتر با پاسخ مثال، رمزین را اسکن کنید.

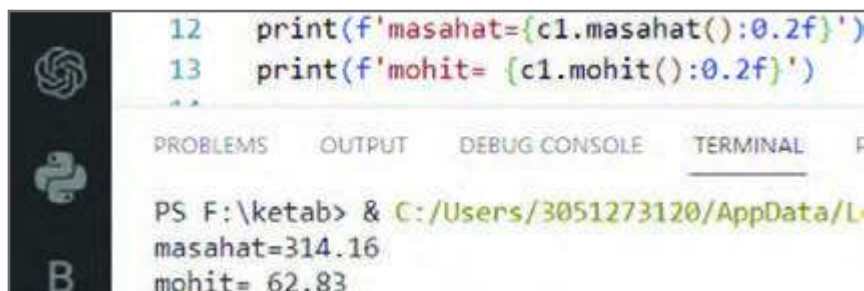
از آنجا که در برنامه از ثابت pi با مقدار ۳/۱۴ استفاده می شود، باید در خط ۱ ماژول math را import کرد. همچنین در خطوط ۲ تا ۱۰ کلاسی به نام Circle به همراه یک تابع __init__ که دارای یک پارامتر r برای دریافت شعاع است ایجاد می شود. در ضمن دو تابع به نام های mohit و masahat برای محاسبه محیط و مساحت تعریف می شود. در خط ۱۱ یک دایره (شیء) به نام c1 از کلاس Circle با شعاع ۱۰ ایجاد شده است. در خطوط ۱۲ و ۱۳ با فراخوانی توابع mohit و masahat از شیء c1 محیط و مساحت آن را چاپ می کند.



خطوط شماره ۱۲ و ۱۳ را توسط f-string تکمیل کنید.

نکته: برای محدود کردن تعداد رقم‌های اعشار عدد چاپ شده کدهای خطوط ۱۲ و ۱۳ باید به صورت شکل زیر تغییر

داده شود.



```
12 print(f'masahat={c1.masahat():0.2f}')
13 print(f'mohit= {c1.mohit():0.2f}')
```

PS F:\ketab> & C:/Users/3051273120/AppData/L...
masahat=314.16
mohit= 62.83

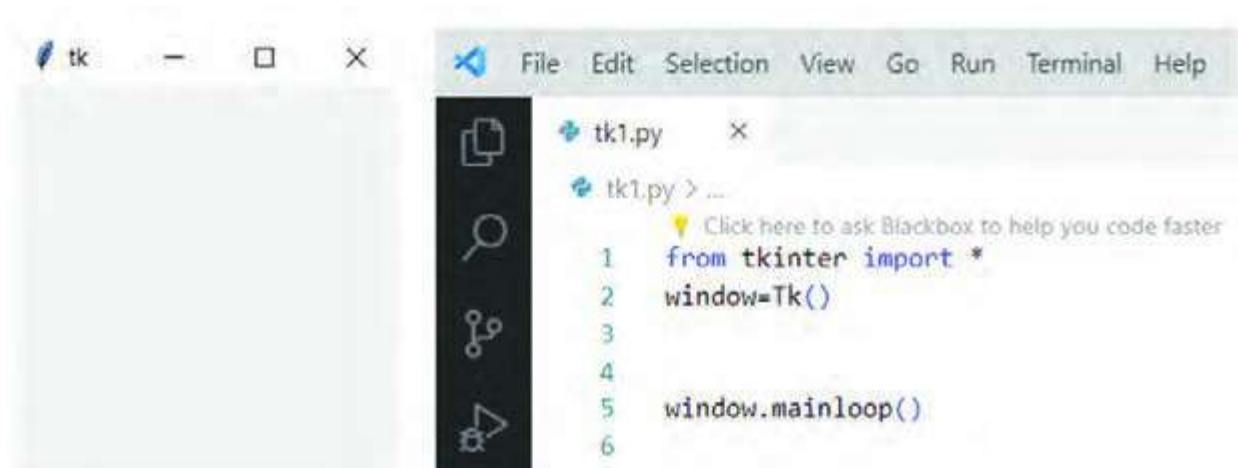
آشنایی با کتابخانه tkinter

تا اینجا خروجی تمام برنامه‌هایی را که بررسی کردیم به صورت متنی بود. امروزه در دنیای برنامه‌نویسی خروجی‌ها متنی نیست و به صورت گرافیکی و یا به عبارت دقیق‌تر تحت ویندوز یا تحت وب است. برای نوشتن برنامه‌های تحت ویندوز از کتابخانه‌هایی مانند tkinter و PyQt5 و Kivy استفاده می‌شود.

برای ایجاد یک پنجره حداقل سه خط کد را به صورت شکل ۷-۴ الف باید نوشت. در خط ۱ تمام محتویات کتابخانه استاندارد tkinter وارد برنامه شده و در خط ۲ یک شیء به نام window از کلاس Tk (یکی از کلاس‌های موجود در ماژول tkinter) ایجاد می‌کنیم. (به جای window می‌توان از هر نام دیگری استفاده کرد.) برای آنکه پنجره به طور مداوم نمایش داده شود در انتهای برنامه (خط ۵) باید متد mainloop را روی window اعمال کرد. حال اگر برنامه اجرا شود، در خروجی پنجره‌ای به شکل ۷-۴ ب نمایش داده می‌شود.



برای آشنایی بیشتر با کتابخانه tkinter، رمزینه را اسکن کنید.



(ب)

(الف)

نحوه تعیین اندازه، عنوان و موقعیت پنجره در صفحه نمایش و تغییر رنگ آن

برای تعیین طول، ارتفاع و محل قرار گرفتن پنجره از متد geometry و برای تغییر رنگ از متد config یا configure استفاده می‌شود. همچنین برای مشخص کردن عنوان صفحه از متد title استفاده می‌شود. برای مثال در شکل ۸-۴ الف کد نمایش پنجره‌ای با عنوان First tkinter، با ابعاد ۴۰۰ در ۳۰۰ پیکسل و در موقعیت ۱۰۰ و ۱۵۰ پیکسل آمده است. همان طور که دیده می‌شود در خط ۵ رنگ پنجره آبی انتخاب شده است.



(ب)



(الف)

شکل ۸-۴

اضافه کردن ابزارک به پنجره

می‌توان به پنجره، ابزارک‌هایی (widget) مانند Button، Label و Entry که به ترتیب برای ورود اطلاعات، نمایش اطلاعات و رویداد کلیک هستند اضافه کرد. برای آشنایی بیشتر با این موضوع به مثال زیر توجه کنید.



برای آشنایی بیشتر با ابزارک‌ها در tkinter، رمزینه را اسکن کنید.

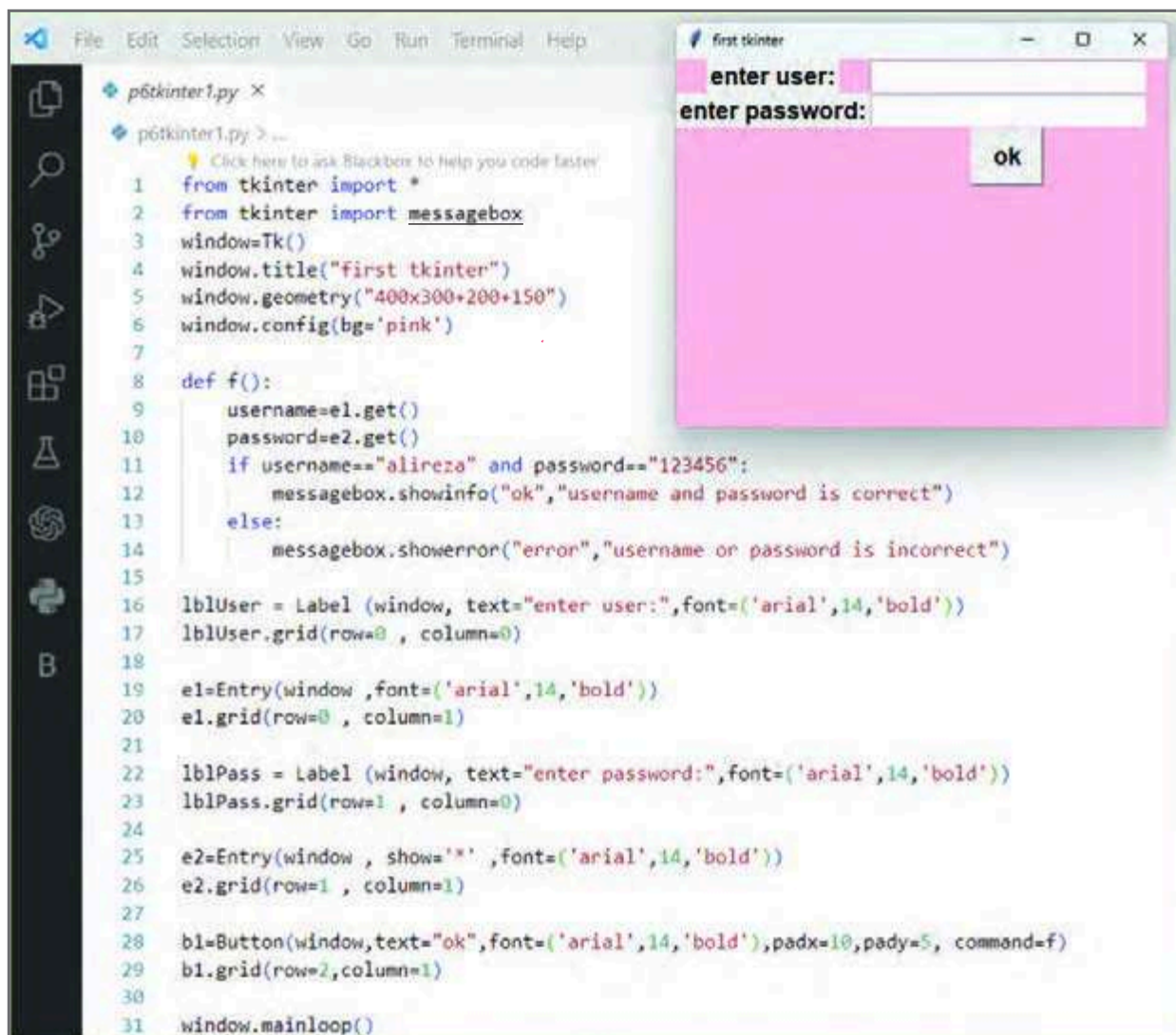
مثال ۷: پنجره‌ای برای دریافت نام کاربری و پسورد طراحی کنید.

پاسخ: تصویری از برنامه مورد نظر در شکل صفحه بعد آمده است که در ادامه هر یک از خطوط این برنامه بررسی شده است. برای آنکه با کلیک روی یک دکمه، تابعی فراخوانی شود، از آپشن command استفاده می‌شود. بنابراین تابعی به نام f برای دریافت نام کاربری و پسورد می‌نویسیم و سپس نام آن را در آپشن command دکمه قرار می‌دهیم که این تابع در برنامه شکل صفحه بعد از خط ۸ تا ۱۴ نوشته شده است و در خط ۲۸ آپشن command با نام تابع مقداردهی شده است.

در خط ۱۶ برنامه یک برجسب (label) به نام lblUser تعریف شده است. سپس در خط ۱۷ توسط متد grid مکان آن روی پنجره مشخص شده که به ترتیب row و column با ° و ° مقداردهی شده است.

در خطوط ۱۹ و ۲۰ یک Entry به نام e1 با row و column به ترتیب 0 و 1 مقداردهی شده‌اند. همچنین در خطوط ۲۲ و ۲۳ یک برجسب به نام lblPass در سطر ۱ و ستون صفر روی پنجره ایجاد شده است.

در خطوط ۲۵ و ۲۶ یک Entry در موقعیت ۱ و ۱ ایجاد شده و توسط آپشن show نوع نمایش رمزه صورت* تبدیل می‌شود. در خط ۲۸ ابزارک دکمه تعریف با آپشن‌های text، font، padx، pady و command تعریف شده است. توجه کنید که آپشن command با نام تابع مقداردهی شده است. در خط ۲۹ در سطر ۲ و ستون ۱ موقعیت دکمه مشخص شده است. همچنین در برنامه زیر از کلاس messagebox برای نمایش کادرهای پیام، کادرهای خطا و... استفاده شده است. همان‌طور که دیده می‌شود در خط ۲ این کلاس وارد شده و سپس در خط‌های ۱۲ و ۱۴ مورد استفاده قرار گرفته‌اند.

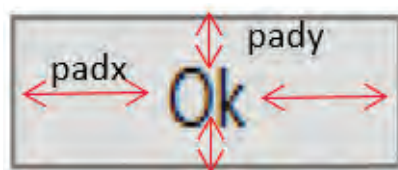


```

1 from tkinter import *
2 from tkinter import messagebox
3 window=Tk()
4 window.title("first tkinter")
5 window.geometry("400x300+200+150")
6 window.config(bg='pink')
7
8 def f():
9     username=e1.get()
10    password=e2.get()
11    if username=="alireza" and password=="123456":
12        messagebox.showinfo("ok","username and password is correct")
13    else:
14        messagebox.showerror("error","username or password is incorrect")
15
16 lblUser = Label (window, text="enter user:",font=('arial',14,'bold'))
17 lblUser.grid(row=0 , column=0)
18
19 e1=Entry(window ,font=('arial',14,'bold'))
20 e1.grid(row=0 , column=1)
21
22 lblPass = Label (window, text="enter password:",font=('arial',14,'bold'))
23 lblPass.grid(row=1 , column=0)
24
25 e2=Entry(window , show='',font=('arial',14,'bold'))
26 e2.grid(row=1 , column=1)
27
28 b1=Button(window,text="ok",font=('arial',14,'bold'),padx=10,pady=5, command=f)
29 b1.grid(row=2,column=1)
30
31 window.mainloop()

```

نکته: توسط آپشن‌های padx و pady یا آپشن‌های width و height می‌توان طول و عرض دکمه را تغییر داد.

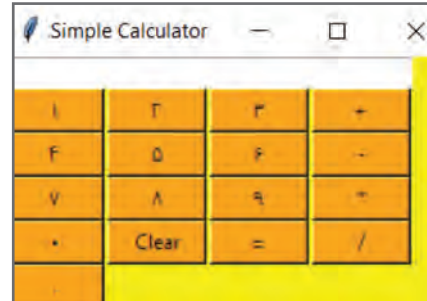




در شکل زیر برنامه یک ماشین حساب ساده نوشته شده است. در گروه خود

عملکرد هر یک از خطوط برنامه را بررسی کنید. نتیجه را به کلاس ارائه دهید. برای آشنایی با برنامه ماشین حساب، رمزینه را اسکن کنید.

```
calculator.py
calculator.py
Click here to ask BlackBox to help you code better.
1 from tkinter import *
2 expression = ""
3 def press(num):
4     global expression
5     expression = expression + str(num)
6     equation.set(expression)
7 def equalpress():
8     try:
9         global expression
10        total = str(eval(expression))
11        equation.set(total)
12        expression = ""
13    except:
14        equation.set(" error ")
15        expression = ""
16 def clear():
17     global expression
18     expression = ""
19     equation.set("")
20 if __name__ == "__main__":
21     gui = Tk()
22     gui.configure(background="light green")
23     gui.title("Simple Calculator")
24     gui.geometry("270x150")
25     equation = StringVar()
26     expression_field = Entry(gui, textvariable=equation)
27     expression_field.grid(columnspan=4, ipadx=70)
28     button1 = Button(gui, text=' 1 ', fg='black', bg='orange', command=lambda: press(1), height=1, width=7)
29     button1.grid(row=1, column=0)
30     button2 = Button(gui, text=' 2 ', fg='black', bg='orange', command=lambda: press(2), height=1, width=7)
31     button2.grid(row=1, column=1)
32     button3 = Button(gui, text=' 3 ', fg='black', bg='orange', command=lambda: press(3), height=1, width=7)
33     button3.grid(row=1, column=2)
34     button4 = Button(gui, text=' 4 ', fg='black', bg='orange', command=lambda: press(4), height=1, width=7)
35     button4.grid(row=1, column=3)
36     button5 = Button(gui, text=' 5 ', fg='black', bg='orange', command=lambda: press(5), height=1, width=7)
37     button5.grid(row=2, column=0)
38     button6 = Button(gui, text=' 6 ', fg='black', bg='orange', command=lambda: press(6), height=1, width=7)
39     button6.grid(row=2, column=1)
40     button7 = Button(gui, text=' 7 ', fg='black', bg='orange', command=lambda: press(7), height=1, width=7)
41     button7.grid(row=2, column=2)
42     button8 = Button(gui, text=' 8 ', fg='black', bg='orange', command=lambda: press(8), height=1, width=7)
43     button8.grid(row=2, column=3)
44     button9 = Button(gui, text=' 9 ', fg='black', bg='orange', command=lambda: press(9), height=1, width=7)
45     button9.grid(row=3, column=0)
46     button0 = Button(gui, text=' 0 ', fg='black', bg='orange', command=lambda: press(0), height=1, width=7)
47     button0.grid(row=3, column=1)
48     plus = Button(gui, text=' + ', fg='black', bg='orange', command=lambda: press("+"), height=1, width=7)
49     plus.grid(row=3, column=2)
50     minus = Button(gui, text=' - ', fg='black', bg='orange', command=lambda: press("-"), height=1, width=7)
51     minus.grid(row=3, column=3)
52     multiply = Button(gui, text=' * ', fg='black', bg='orange', command=lambda: press("*"), height=1, width=7)
53     multiply.grid(row=4, column=0)
54     divide = Button(gui, text=' / ', fg='black', bg='orange', command=lambda: press("/"), height=1, width=7)
55     divide.grid(row=4, column=1)
56     equal = Button(gui, text=' = ', fg='black', bg='orange', command=equalpress, height=1, width=7)
57     equal.grid(row=4, column=2)
58     clear = Button(gui, text=' Clear ', fg='black', bg='orange', command=clear, height=1, width=7)
59     clear.grid(row=4, column=3)
60     Decimal = Button(gui, text=' . ', fg='black', bg='orange', command=lambda: press('.'), height=1, width=7)
61     Decimal.grid(row=5, column=0)
62     gui.mainloop()
```

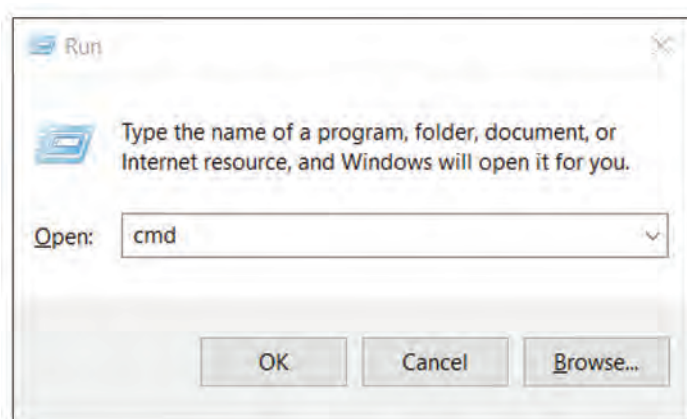


نحوه ایجاد خروجی exe از برنامه

برای اجرای برنامه‌هایی که تا اینجا نوشته شد، باید وارد محیط IDLE یا Visual Studio Code شده و از یکی از دو روش زیر آن را اجرا کرد:

الف) کلید F5 ب) منوی Run

برای اجرای برنامه به صورت یک نرم افزار تحت ویندوز و بدون نیاز به IDLE یا Visual Studio Code لازم است از دستور pip ماژول pyinstaller نصب شود. به این منظور ابتدا از طریق گزینه Run در ویندوز دستور cmd را تایپ و اجرا می کنیم (شکل ۴-۹).

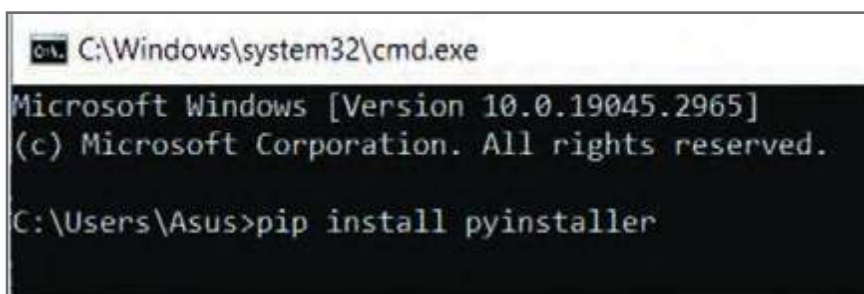


شکل ۴-۹



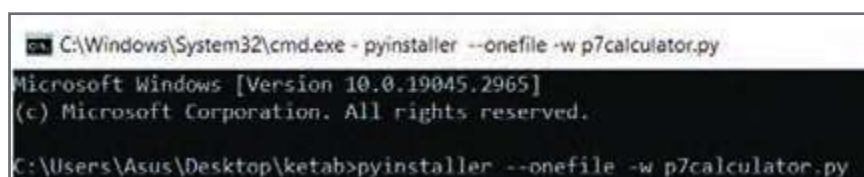
برای آشنایی بیشتر با نحوه ایجاد خروجی exe از برنامه، رمزینه را اسکن کنید.

پس از ورود به محیط command دستور pip install pyinstaller را می نویسیم تا ماژول pyinstaller نصب شود (شکل ۴-۱۰).



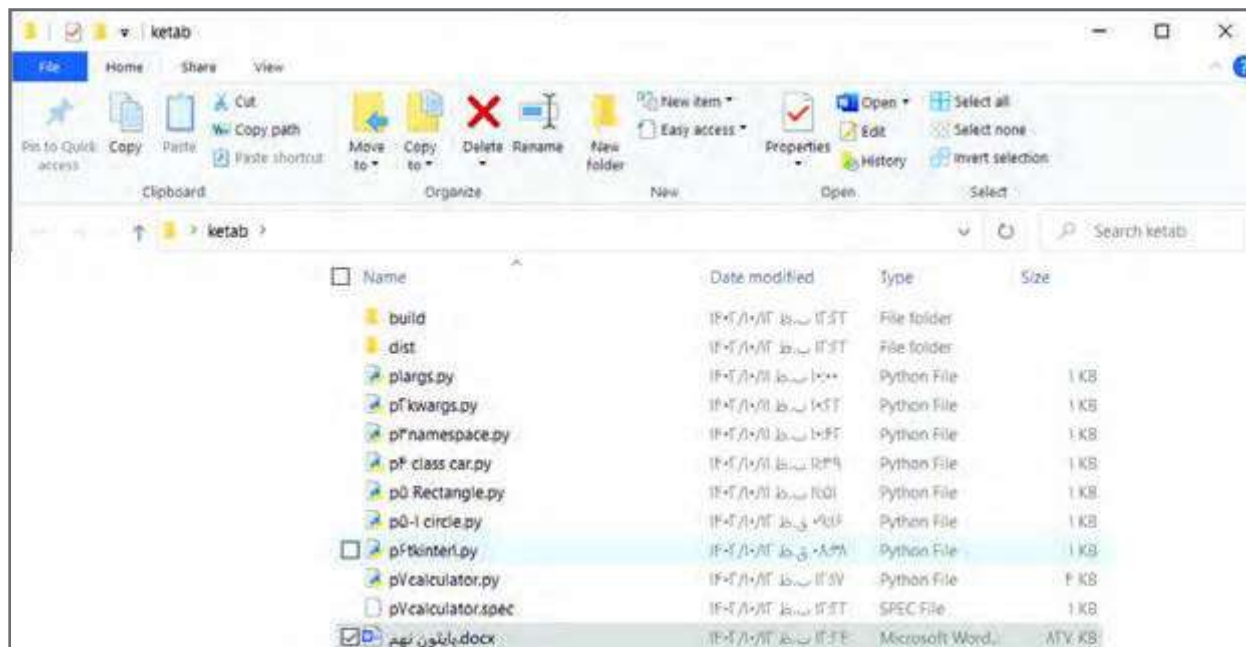
شکل ۴-۱۰

پس از اینکه ماژول pyinstaller نصب شد در مسیری که برنامه پایتونی (calculator.py) قرار دارد وارد می شویم. به طور مثال در دسکتاپ فولدری به نام ketab وجود دارد که در آن یک برنامه پایتونی قرار دارد. با باز کردن فولدر و تایپ cmd در نوار آدرس پنجره باز شده و زدن کلید enter، از مسیر فولدر ketab وارد cmd شده و دستور آمده در شکل ۴-۱۱ را تایپ می کنیم.



شکل ۴-۱۱

حال در کنار برنامه یک فولدر به نام dist ساخته می‌شود که در آن یک فایل exe وجود دارد. با دابل کلیک روی آن بدون نیاز به IDLE یا VSCode برنامه ما از طریق محیط ویندوز اجرا می‌شود.



در کتاب‌های کار و فناوری دوره اول متوسطه در پایه‌های هفتم، هشتم و نهم پودمان‌های زیادی در قالب پروژه‌های طراحی و ساخت، پروژه‌های پرورش و نگهداری و پروژه‌های تعمیر و نگهداری وجود دارد و تاکنون تعداد زیادی از آن‌ها را فرا گرفته‌اید. در جدول زیر عناوین این پودمان‌ها جهت یادآوری آورده شده است.

عناوین پودمان‌های کتاب کار و فناوری پایه‌های هفتم، هشتم و نهم

پایه هفتم		پایه هشتم		پایه نهم	
بخش اول: پودمان‌های تجویزی	پودمان ۱: نوآوری و فناوری	بخش اول: پودمان‌های تجویزی	پودمان ۱: شهروند الکترونیکی اتصال به شبکه، پست الکترونیکی	بخش اول: پودمان‌های تجویزی	پودمان ۱: الگوریتم
	پودمان ۲: کاربرد فناوری اطلاعات و ارتباطات		پودمان ۲: شهروند الکترونیکی ۲		پودمان ۲: ترسیم با رایانه
	پودمان ۳: جست‌وجو و جمع‌آوری اطلاعات		پودمان ۳: برنامه‌نویسی پایتون (۲)		پودمان ۳: ساز و کارهای حرکتی
	پودمان ۴: مستندسازی		پودمان ۴: الکترونیک		پودمان ۴: برنامه‌نویسی پایتون (۳)
	پودمان ۵: اشتراک‌گذاری اطلاعات		پودمان ۵: کار با فلز		پودمان ۵: هدایت تحصیلی – حرفه‌ای
بخش دوم: پودمان‌های نیمه‌تجویزی	پودمان ۶: برنامه‌نویسی	بخش دوم: پودمان‌های نیمه‌تجویزی	پودمان ۶: صنایع دستی (بافت)	بخش دوم: پودمان‌های نیمه‌تجویزی	پودمان ۶: برق
	پودمان ۷: کسب و کار		پودمان ۷: پرورش و نگهداری از حیوانات		پودمان ۷: تأسیسات مکانیکی
	پودمان ۸: نقشه‌کشی		پودمان ۸: امور اداری و مالی		پودمان ۸: عمران
	پودمان ۹: کار با چوب		پودمان ۹: معماری و سازه (ماکت‌سازی)		پودمان ۹: خودرو
	پودمان ۱۰: پرورش و نگهداری گیاهان				پودمان ۱۰: پایش رشد و تکامل کودک
بخش دوم: پودمان‌های نیمه‌تجویزی	پودمان ۱۱: پوشاک	بخش دوم: پودمان‌های نیمه‌تجویزی		بخش دوم: پودمان‌های نیمه‌تجویزی	پودمان ۱۱: صنایع دستی (برجسته‌کاری روی فلز مس)
	پودمان ۱۲: خوراک				

با یادآوری تجربیاتی که هنگام انجام این پروژه‌ها کسب نموده‌اید، تا حدودی به میزان استعداد، توانایی و علاقه خود نسبت به آن مهارت‌ها پی برده‌اید. استعداد، توانایی و علاقه، بخشی از ویژگی‌های شخصیتی شما هستند که به انتخاب رشته تحصیلی – حرفه‌ای شما کمک فراوانی می‌کنند.